JUCE and Params

Intro to Audio Tech 2
Jocelyn Kavanagh
Tuesday, April 7

Georgia Tech

Lesson Outline

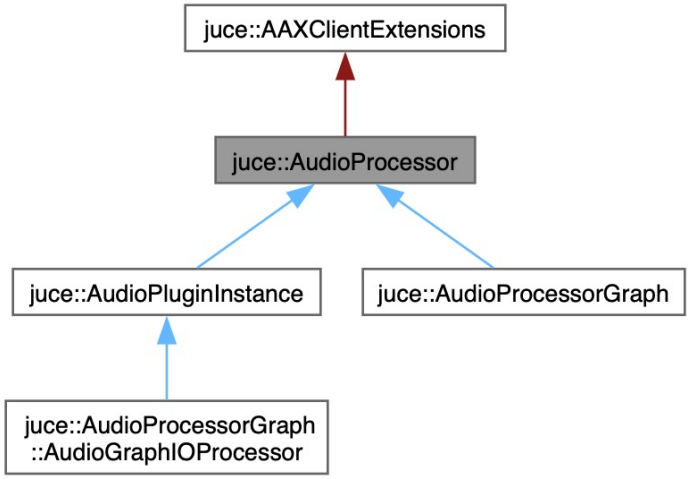
- JUCE at a high level
- Continuous envelope
- LFO
- GUIs

Georgia Tech

JUCE System Flow

JUCE Audio Processor

- Last week, we edited the AudioProcessor class, but we didn't really delve in
- Here is a quick overview of the class and the functions that "come with it"



prepareToPlay()

- Set-up and reset section, where we set our block size and sampling rate given from the DAW
- Block size can vary
  - DAW will give the Max expected block size
  - We generally want to add a bit of wiggle room for ourselves in case of real-time change
- Called on launch and any time it is reset to start audio processing

processBlock()

- Audio callback
- Where the audio processing actually happens
- Gives you the buffer to edit from input that will become the output buffer
- Also gives you the midi messages from the current block cycle
- Any code in the processBlock must be complete in less time than the length of the incoming buffer it is processing
- This block is called continuously while the user is playing audio

What can’t happen in processBlock()?

- We cannot
  - do anything that takes longer than our block size length
  - Do an operation with an unknown or unbounded execution time
- No allocation
  - Don’t allocate or deallocate memory (buffers) in the Audio thread
  - This requires us to use the operating system calls which we can’t control the prioritization or execution time
  - JUCE handles most of this for us. We should do any additional allocation in prepareToPlay()
- No I/O
  - Don’t read/write from a file, log, or network
  - This requires us to use the operating system calls which we can’t control the prioritization or execution time
  - Do any reading ahead in prepareToPlay() or create a new background thread

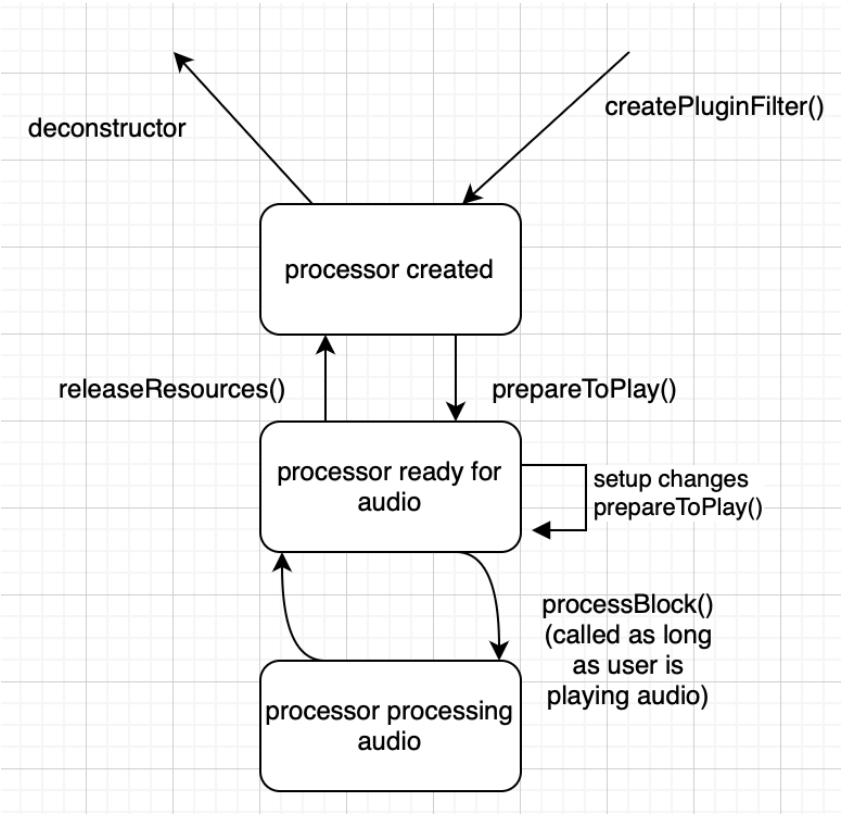
JUCE Audio Processor

- releaseResources()
  - Frees up space when your application is not in use
- isBusLayoutSupported()
  - Says whether or not your plugin supports certain layouts (mono, stereo, 5:1, etc)
- createEditor and hasEditor
  - Makes the GUI
- getStateInformation and setStateInformation
  - Set and load “presets” or states for when you close and re-open the plugin
- createPluginFilter()
  - Actually creates the plugin based on our targets (VST3, AU, etc)

JUCE Audio Processor

- AudioProcessor() and ~AudioProcessor()
  - Constructor and destructor
  - Create the application and set-up initial I/O, then destroy plugin on close
- getTailLengthSeconds()
  - This keeps your application running for X amount of seconds on close (delay or reverb)
- getNumPrograms, getCurrentProgram, getProgramName, changeProgramName
  - These all deal with plugin presets

JUCE Program Flow



Implement Envelope and LFO

Georgia Tech

Audio Params and Value Tree

Audio Parameter Value Tree State

- A system in JUCE for managing plugin parameters safely and consistently
- Stores all plugin parameters in one place
- Keeps parameters thread-safe (audio thread vs UI thread)
- It allows us to
  - connect seamlessly to our DAW
  - easily save states
  - Keep our parameters organized under one structure
  - quickly build a GUI